Miniprojeto ADS

Medição ativa com iperf com apoio de medição passiva(tcpdump)

26 de março de 2026

Igor Budag, Ricardo Wagner

Sumário

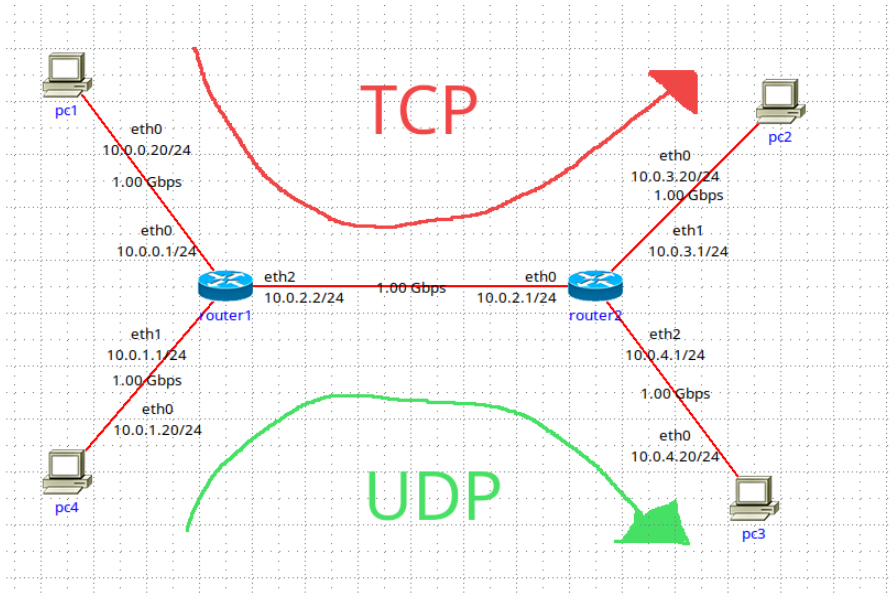
1 Objetivo 3
2 Variáveis 6
3 Metodologia 10
4 Resultados 16
5 Conclusão 20

1 Objetivo

Objetivo

A proposta desse miniprojeto consiste em analisar uma situação de transmissão TCP com uma carga UDP intensa passando simultaneamente pelo mesmo link (congestionamento), verificando qual algoritmo TCP (Reno ou Cubic) se sai melhor. Para análise, foi feito um experimento que terá parâmetros fixados, variáveis de saída e também fatores que serão variados. O experimento deve ser realizado com 8 repetições para cada configuração e deve se fazer todas as combinações possíveis de fatores. No nosso caso 2^2, já que temos 2 fatores.

Objetivo


TCP
UDP
pc1
eth0
10.0.0.20/24
1.00 Gbps
eth0
10.0.0.1/24
eth2
10.0.2.2/24
1.00 Gbps
eth0
10.0.2.1/24
router1
router2
eth1
10.0.1.1/24
1.00 Gbps
eth0
10.0.1.20/24
pc4
pc2
eth0
10.0.3.20/24
1.00 Gbps
eth1
10.0.3.1/24
eth2
10.0.4.1/24
1.00 Gbps
eth0
10.0.4.20/24
pc3

2 Variáveis

Variáveis de Saída

Vazão Média do Fluxo TCP (iperf)

Eficiência do Fluxo TCP (tshark)

Variáveis de Entrada (fatores)

Algoritimo TCP	Reno/Cubic
BER no enlace entre roteadores	$10^{-6}/10^{-5}$

Parâmetros Fixados

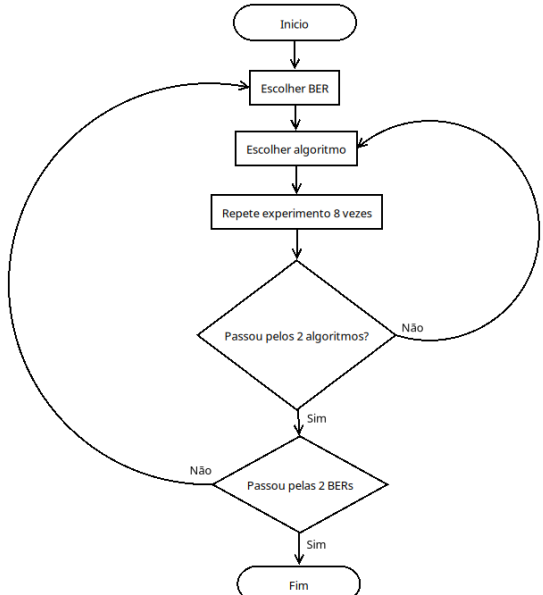
Enlace Ethernet	1 [Gbps]
Janela TCP default	128 [KByte]
Duração do teste	30 [s]
Carga UDP	900 [Mbps]
Número de repetições	32

3 Metodologia

Metodologia

Neste trabalho, foi utilizado o software IMUNES para a criação da simulação da rede. Além disso, empregamos a linguagem Python para automatizar os processos de coleta e análise de dados. Essas duas funções foram organizadas em códigos distintos: o script.py, responsável pela coleta, e o analise.py, dedicado à análise dos dados.

Loop Principal



Cálculo da eficiência e set da BER

- Tentamos utilizar o comando tshark disponibilizado para o cálculo da eficiência e vlink para alterar a BER, mas não obtivemos resultados satisfatórios.
- No caso da BER, ela não alterava, então buscamos inserir uma perda em porcentagem equivalente a essa BER.
- Já para o cálculo da eficiencia, fizemos uma busca utilizando regex no próprio pcap.
- Vamos mostrar os trechos de código a seguir:

Set da BER

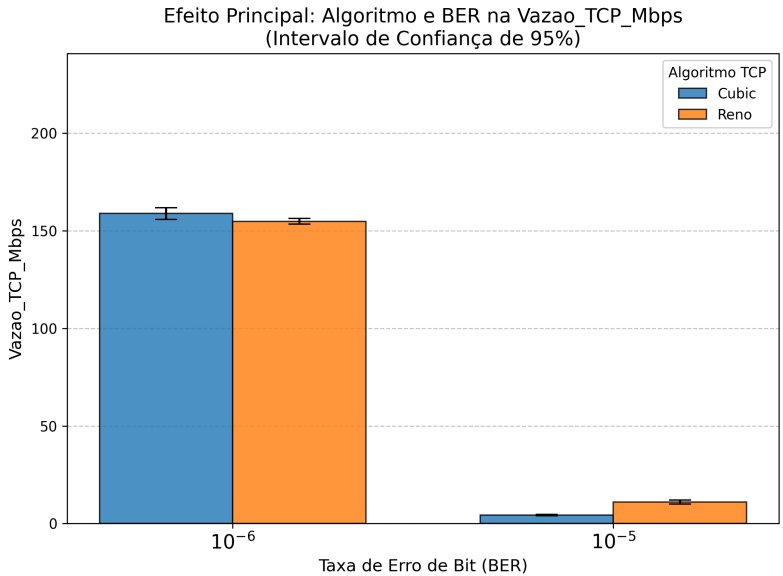

```python
def set_packet_loss(ber):
    # Converte a BER em taxa de perda de pacotes (%) e aplica direto no kernel do PC4.
    if ber == 0:
        loss_percent = 0.0
        ber_texto = "0"
    else:
        prob_sucesso_pacote = (1 - ber) ** 12000
        prob_perda_pacote = 1 - prob_sucesso_pacote
        loss_percent = prob_perda_pacote * 100
        ber_texto = f"10^{int(math.log10(ber))}" # Transforma 1e-05 em 10^-5

    # 1. Apaga qualquer regra de controle de tráfego antiga na interface eth0 do PC4
    run("sudo himage pc4 tc qdisc del dev eth0 root 2>/dev/null")

    # 2. Aplica a nova regra de perda de pacotes
    if loss_percent > 0:
        run(f"sudo himage pc4 tc qdisc add dev eth0 root netem loss {loss_percent:.4f}%")
```

Cálculo da eficiencia


```python
def process_pcap(filename):
    tcp_output = run(f"sudo himage pc4 tcpdump -nne -r {filename}")

    bytes_tcp = 0
    bytes_totais = 0

    for linha in tcp_output.splitlines():
        match = re.search(r'length\s+(\d+):.*length\s+(\d+)', linha)
        if match:
            tamanho_frame = int(match.group(1))
            tamanho_payload = int(match.group(2))

            bytes_tcp += tamanho_payload
            bytes_totais += tamanho_frame

    eficiencia = bytes_tcp / bytes_totais if bytes_totais > 0 else 0
    return bytes_tcp, bytes_totais, eficiencia, tcp_output
```

4 Resultados

Vazão Média


Efeito Principal: Algoritmo e BER na Vazao_TCP_Mbps
(Intervalo de Confiança de 95%)
Algoritmo TCP
Cubic
Reno
Vazao_TCP_Mbps
0
50
100
150
200
10^{-6}
10^{-5}
Taxa de Erro de Bit (BER)

Eficiência

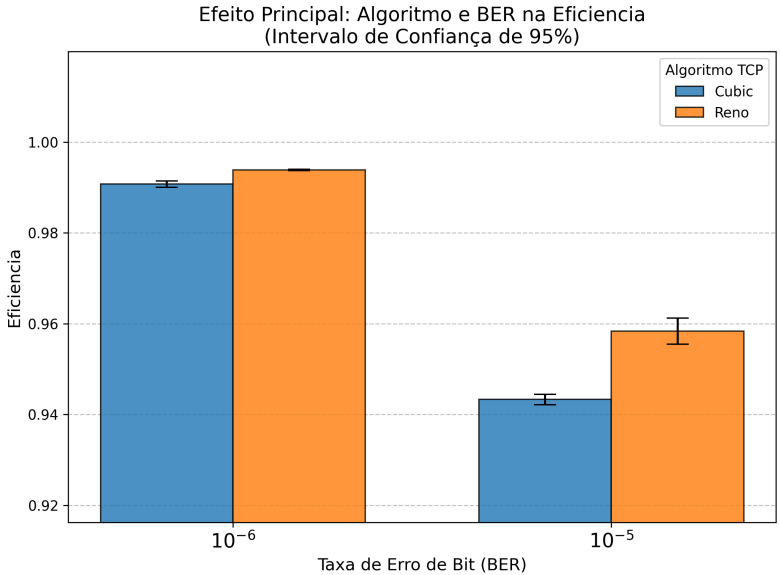

Efeito Principal: Algoritmo e BER na Eficiencia
(Intervalo de Confiança de 95%)
Algoritmo TCP
Cubic
Reno
1.00
0.98
0.96
0.94
0.92
Eficiencia
10^{-6}
10^{-5}
Taxa de Erro de Bit (BER)

Tabela Resumo

Algoritmo	BER	Vazão Média (Mbps)	Eficiência Média
Cubic	10^{-6}	158.88 ± 3.05	0.9908 ± 0.0007
Cubic	10^{-5}	4.38 ± 0.33	0.9433 ± 0.0012
Reno	10^{-6}	154.88 ± 1.44	0.9939 ± 0.0001
Reno	10^{-5}	11.09 ± 1.00	0.9584 ± 0.0028

5 Conclusão

Conclusão

Pelos resultados obtidos podemos observar que quando a rede está sem problemas, os algoritmos não tem uma diferença significativa de desempenho, mas quando temos uma BER pior, podemos ver que o Reno é superior em ambas métricas.